ESCAPE ROOM · JOGOS MORTAIS · JIGSAW OS

Guia do Nível 3 — A Prova Final

Gerado de TESTES-NIVEL3.md em 31/10/2026

> ⚠ **SPOILER.** Este documento tem todas as respostas. É só para testadores e para o mestre do jogo, nunca deixe os jogadores verem.

Todos os valores abaixo vêm de `js/level3-data.js`. Se alguém mudar esse arquivo, este guia precisa ser atualizado.

A ideia do nível: é o mais difícil. As questões **principais** exigem o que se aprende até o 3º semestre de Engenharia de Computação (programação em C, estruturas de dados e circuitos digitais). Cada principal tem um **opcional** que **não exige conhecimento prévio** e, de prêmio, dá uma **ajuda** de como resolver a principal. O que está fora do curso (ASCII, Morse) vem explicado nos papéis.

---

1. Montagem

O quê	Onde	Como
Jogo	Notebook da sala	Abrir `index.html` no Chrome e escolher **Nível 3** (ou Ctrl + Shift + 3)
Projetor	PC do mestre do jogo, ligado ao projetor	Abrir `projetor.html`, escolher **Nível 3** e apertar **F** (tela cheia)
Papéis	Sala	Imprimir `imprimir-nivel3.html` (Ctrl + P, A4)

O que fazer com cada papel:

- **Página 1, nós da lista (10 cartões):** recortar e esconder cada um num ponto diferente da sala. 4 são iscas.
- **Página 2, circuito do registrador:** colar na parede.
- **Página 3, tabela ASCII:** colar na parede (precisa estar visível desde o começo, para o login).
- **Página 4, código Morse:** colar na parede ou deixar numa mesa.
- **Página 5, manual da fonte (energia.exe):** esconder, por exemplo numa gaveta ou pasta. Sem ele o `energia.exe` não tem solução.

---

2. Resposta e cola rápida

Resposta final: `SEGFAULT`**.** O grupo fala o kill switch para o mestre do jogo pela câmera.

Etapa	Resposta	De onde vem
Login	`PONTEIRO`	Dump `53 45 4E 48 41 3A 50 4F 4E 54 45 49 52 4F` = "SENHA:PONTEIRO" (tabela ASCII)
Q1 Lista encadeada	`MALLOC`	`head = 0x2C` no `lista.c` + cartões na sala

Etapa	Resposta	De onde vem
Q2 Display	`DEAD`	Expressão do segmento g no `display.exe` → o display mostra "dEAd"
Q3 Registrador	**110 = 6** → `msg_6`	Circuito na parede + `registrador.log` (início 001, 5 pulsos)
Chave	`MALLOCDEAD`	Lista + display, como diz o `prova_final.txt`
Final	`OKILLSWITCHESEGFAULT`	`decifra.c` consertado, executado com `msg_6.txt` e `MALLOCDEAD`

As 4 primeiras questões depois do login podem ser feitas **ao mesmo tempo**, por pessoas diferentes.

---

3. As questões principais

Login: dump de memória

A tela de login mostra 14 bytes em hexadecimal. Com a tabela ASCII: 53=S, 45=E, 4E=N, 48=H, 41=A, 3A=:, 50=P, 4F=O, 4E=N, 54=T, 45=E, 49=I, 52=R, 4F=O. Depois de 3 erros aparece a dica "Cada par de símbolos é 1 byte em hexadecimal...".

Q1: lista encadeada (sala)

O `lista.c` mostra `head = 0x2C`. Seguindo os cartões:

Endereço	Dado	next
0x2C	M	0x7A
0x7A	A	0x13
0x13	L	0x5E
0x5E	L	0x91
0x91	O	0x3B
0x3B	C	NULL

Iscas (ninguém aponta para elas): 0x68 'R', 0xA0 'E', 0x4F 'X', 0xC7 'F' (esta aponta para o head, mas ninguém chega nela).

Q2: display.exe (mapa de Karnaugh)

O vírus apagou a lógica do segmento g, e o display fica **apagado ("SEM SINAL")** até o grupo gravar a expressão certa. Assim não dá para "imaginar o g aceso" e ler a palavra sem resolver. A folha de dados mostra a coluna de g para 0–F. O grupo digita a expressão de g com **no máximo 12 literais**. **Qualquer expressão equivalente vale.** Uma resposta mínima (11 literais):

```
A'BC' + AB' + B'C + CD' + AD
```

- Escrita: `+` = ∨ (OU), letras juntas = ∧ (E), `'` = ¬ (NÃO). A tela mostra os símbolos lógicos. Também aceita `∨`, `∧`, `¬A`, `!A`, `~A`, `*`, `&`, `|`, `^` = XOU e parênteses.
- Mais de 12 literais: só avisa "simplifique" (não é erro).
- Expressão errada: diz em quantas das 16 entradas não bate e conta como **erro**.

- Com a expressão certa, o display mostra **dEAd** → DEAD.

Q3: registrador (sala)

`D2 = Q1`, `D1 = Q0`, `D0 = Q2 XOR Q1`. Todos mudam ao mesmo tempo.

Pulso	Q2 Q1 Q0
0	0 0 1
1	0 1 0
2	1 0 1
3	0 1 1
4	1 1 1
5	**1 1 0 = 6**

O arquivo é o **msg_6**. Os outros 7 arquivos, com a chave certa, também dizem "O KILL SWITCH E ..." com outra palavra (NULLPOINTER, STACKOVERFLOW, DEADLOCK, MEMORYLEAK, BUFFEROVERFLOW, RECURSAO, KERNELPANIC). Então **não dá para chutar** sem o registrador: quem testar todos terá 8 respostas e cada erro custa 5 min.

Final: decifra.c (programação em C de verdade)

Só a função `decifra()` pode ser editada. O código roda num **interpretador de C de verdade** (`js/minic.js`): erros de compilação com linha e coluna, `Segmentation fault` ao escrever fora do vetor, lixo de memória, `%` como no C, `printf` para depurar e limite de tempo para loop infinito.

A função tem **5 bugs**:

#	Linha	Bug	Correção
1	8	Falta `;` depois de `strlen(chave)` (erro de compilação)	`int k = strlen(chave);`
2	9	`i <= n` passa do fim da string	`i < n`
3	11	`chave[i % n]` usa o tamanho errado (embaralha a mensagem inteira)	`chave[i % k]`
4	12	`(c - d) % 26` fica negativo em C	`(c - d + 26) % 26`
5	depois do for	Falta o `'\0'` (a saída termina com lixo)	`saida[n] = '\0';`

Função corrigida:

```
void decifra(char *msg, char *chave, char *saida) {
    int n = strlen(msg);
    int k = strlen(chave);
    for (int i = 0; i < n; i++) {
        int c = msg[i] - 'A';
        int d = chave[i % k] - 'A';
        saida[i] = (c - d + 26) % 26 + 'A';
    }
    saida[n] = '\0';
}
```

Qualquer outra solução que funcione também vale (por exemplo, com ponteiros). Para executar: escolher `msg_6.txt`, digitar `MALLOCDEAD` e clicar em **Compilar e executar**. A saída é `OKILLSWITCHESEGFAULT` = "O KILL SWITCH É SEGFAULT", e o J diz "Agora digam para mim". O botão ↺ **Restaurar original** volta o código do começo.

O que se vê com bugs: só com o 1 consertado, a saída sai embaralhada a partir da 11ª letra e com lixo no fim. Os bugs 4 e 5 sozinhos deixam a palavra quase legível, mas o 3 impede o chute.

---

4. Opcionais: a ajuda de cada questão

Nenhum opcional exige conhecimento prévio, mas todos são **difíceis de propósito**: custam tempo e trabalho em equipe. Ao vencer, a ajuda aparece na janela e fica guardada em `ajudas.txt` (ícone que aparece na primeira ajuda).

Opcional	Como vencer	Ajuda que libera
`radio.exe`	Morse **rápido** (som + lâmpada) de uma frase inteira: **SIGA O NEXT ATE O NULL** (17 letras). Tabela Morse na sala. Espaços e maiúsculas não importam. Dá para ouvir quantas vezes quiser	Como ler uma lista encadeada (Q1)
`placa.exe`	Quebra-cabeça de trilhas 6×6: clicar gira a peça (botão direito gira ao contrário). Todas as 36 peças precisam ficar ligadas ao chip do centro, **sem pontas soltas** (o contador mostra as duas coisas). A placa é sorteada uma vez por partida (fechar e reabrir a janela mantém o progresso). **A cada 50 s o J gira 2 peças sozinho**	Mapa de Karnaugh de g já preenchido + passos de como agrupar + um exemplo resolvido (a linha AB = 10 → `AB'`, 1 dos 5 termos) (Q2)
`energia.exe`	Painel de fios: na tela os pinos só têm **número**. O **manual está na sala** (página 5): cor → sinal → número do pino. **Um curto desliga todos os fios** e conta como erro. Resposta: Amarelo→3, Vermelho→6, Laranja→5, Preto→2, Verde→4, Roxo→1	Como funciona o flip-flop D + os 2 primeiros pulsos (Q3)
`invasao.exe`	Terminal hacker **sem dicas** e com rastreamento de **75 s** (derruba a conexão; é preciso conectar de novo). Caminho: `help`, `scan`, `crack 192.168.6.66`, `connect 192.168.6.66`, `ls`, `decrypt chave.enc`	A lista dos 5 bugs do `decifra.c`, sem a correção pronta (Final)

Outros arquivos:

- `diario.txt`: a história do J (trancou o curso no 3º semestre por causa de um segfault). Só história.

- **Lixeira** → `kill_switch.exe`**: armadilha**, dá jump scare sempre (clique duplo).

Erros (Morse errado, curto-circuito, expressão errada) passam por `Interference.error`: o 1º erro nunca dá susto, há 90 s de descanso depois de um susto e a chance cresce 30% por erro (veja `scareRules` no `js/config.js`). Erros de compilação e de execução no `decifra.c` **não** dão susto: errar faz parte de programar.

---

5. Mestre do jogo: projetor e resposta

Tecla (`projetor.html`, Nível 3)	O que faz
Espaço	Inicia ou pausa o cronômetro ("O VÍRUS APAGA TUDO EM 30:00")
P	**Kill switch errado:** toca a cena do susto final (a mesma do Nível 1), tira **5 min** e mostra "Tentem de novo"
V	**Certo:** "CORRETO. O kill switch era SEGFAULT. Vocês passaram na prova que eu não passei."
D / Shift + D	+1 / −1 dica usada
↑ / ↓	+1 / −1 minuto
R (2 vezes seguidas)	Reinicia tudo
F / H	Tela cheia / esconde o painel de teclas
Ctrl + Shift + 1/2/3	Troca de nível

Se o grupo falar outra palavra de um `msg_N` errado (DEADLOCK, RECURSAO...), aperte **P**.

Atalhos no notebook: Ctrl + Shift + X reinicia; **Ctrl + Shift + L** mostra o tempo de cada etapa (login, display, decifra, ajudas...); **Ctrl + Shift + H** dispara a interferência do hacker.

---

6. Dicas prontas (1 por jogador, pedidas pela câmera)

Antes de dar dica, lembre o grupo de que **os programas menores dão ajuda**. Aperte **D** no projetor a cada dica dada.

Onde o grupo travou	Dica leve	Dica direta
Login	"Esses números são letras disfarçadas."	"Cada par é um byte em hex. Usem a tabela ASCII da parede."
Lista	"O head é só o começo."	"Achem o cartão 0x2C e sigam o next até o NULL."
Display	"Nem toda letra precisa aparecer em todo termo."	"Façam o mapa de Karnaugh de g: são 5 grupos."
Registrador	"Os 3 flip-flops mudam juntos, com os valores de antes."	"Façam uma tabela pulso a pulso. Começa em 001."

Onde o grupo travou	Dica leve	Dica direta
Chave	"O prova_final.txt diz o que vai na chave."	"Lista + display, tudo junto: 10 letras."
decifra.c (compilação)	"Leiam a mensagem do compilador: ela diz a linha."	"Falta um ; na linha 8."
decifra.c (saída estranha)	"A chave tem quantas letras? E a mensagem?"	"chave[i % k], i < n, + 26 antes do %, e o '\0' no fim."
Arquivo	"Todos os arquivos dizem 'kill switch'. Só um diz a verdade."	"O número do arquivo é o estado final do registrador."

7. Pular etapas (console do navegador, F12)

```
CONFIG.bootSpeed = 20      // boot rápido (antes de ligar)
OS.openApp('decifra')      // abrir qualquer app pelo id: prova, lista, display, registrador,
mensagens, decifra,
                           // radio, placa, energia, invasao, diario, ajudas, lixeira
```

8. O que observar nos testes

- **Tempo:** a meta é perto de 30 min. É o nível mais difícil: se ninguém terminar, aumente com `projetor.html?nivel=3&min=35`.
- **Divisão:** o grupo se divide entre lista (sala), display, registrador (sala) e C?
- **Opcionais:** quem não sabe a matéria procura os opcionais? As ajudas bastam para destravar?
- **C:** quanto tempo o grupo leva para achar os 5 bugs? Alguém usa `printf` para depurar?

9. Ainda provisório

- **Vídeos dos sustos:** lugares `nivel3` e `nivel3_armadilha` em `scareMedia` (`js/config.js`).
- **Sons:** todos ainda vazios.